Student: Paul Runov

Course: ECE 3210

Subject: Lab 6, Aliasing

Date: April 15, 2026

1 Introduction

The purpose of this lab was to become intimately familiar with the workings of the FFT (Fast Fourier Transform) algorithm.

First the DFT (Discrete Fourier Transform) is created. Then the FFT is built from scratch, which is the basis for other functions that implement the FFT. The functions that build on top of the FFT or the Inverse FFT (using the forward FFT), and the frequency-domain convolution implementation. The time-domain convolution is also created to see how it stacks up against the frequency-domain convolution.

2 Theory

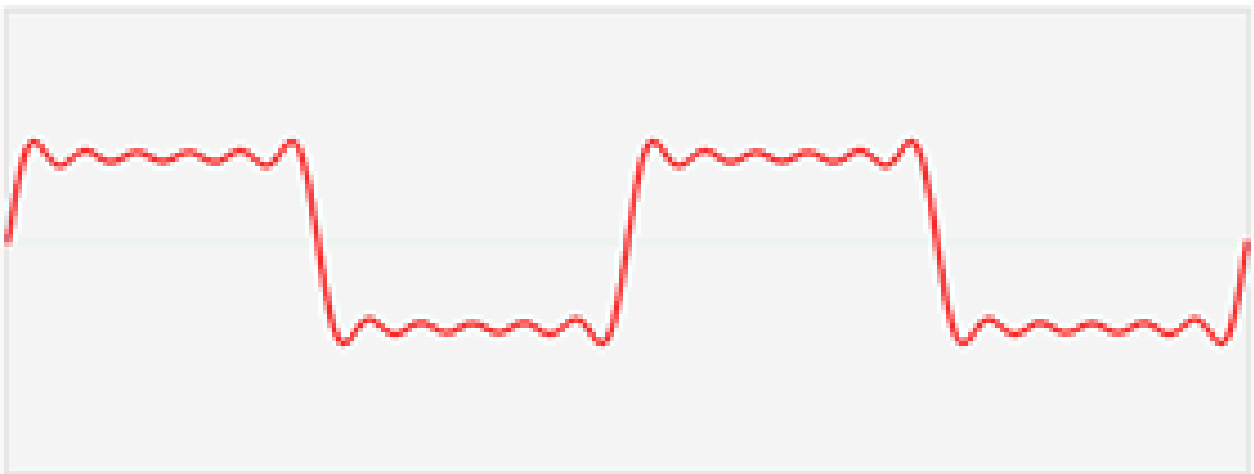
2.1 DFT

The DFT is technically the basis for this lab, since the FFT is just a faster implementation of the DFT. The DFT is what is used to transform a discrete set of data points into the frequency domain. The formula for it is shown below.

$$F(r) = \sum_{k=0}^{N_0-1} x_k \cdot e^{-j\frac{2\pi}{N_0}rk} \tag{1}$$

When the points are in the frequency domain, they give important information about how much of a specific frequency exists in that signal.

The example below is a visualization of how the the principle of the $time <-> frequency$ transformation works, and how the amount of each frequency is shown. If it is opened with a pdf viewer that supports graphics, it will show up (such as Adobe Acrobat)



Here is an image to illustrate the time-frequency transform in case the animated illustration doesn't work.

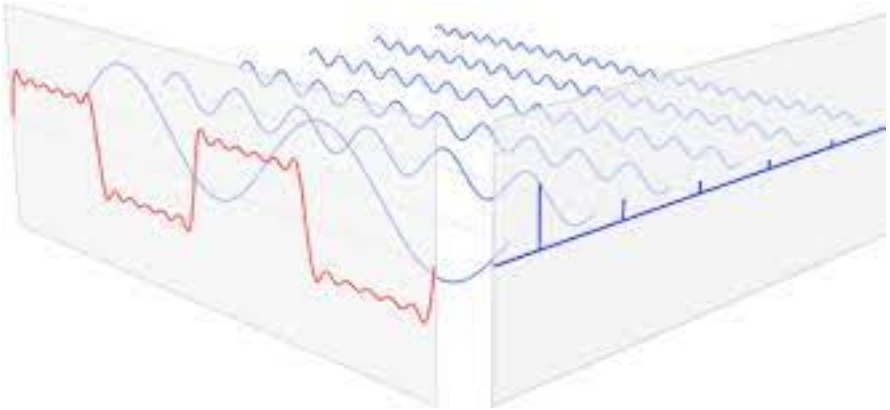


Figure 1: Image of the time $< - >$ frequency domain visualized

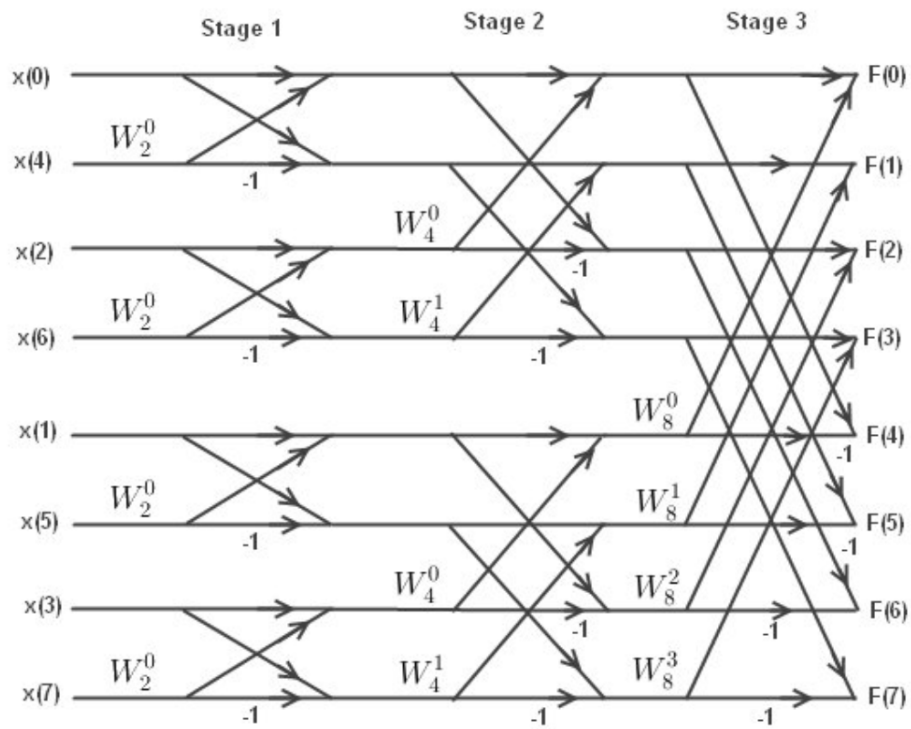


Figure 2: Butterfly Plot

2.2 FFT

The FFT is a greatly improved and optimized way to calculate the DFT of a discrete data-set. A DFT takes $O(N^2)$ time to run, while the FFT takes $O(Nlog(N)$ time.

A classic way to visualize the FFT is with a butterfly diagram. This diagram shows the way every point is calculated from the previous points.

The Fast Fourier Transform (FFT) achieves its efficiency by exploiting the symmetries and periodicities of the Discrete Fourier Transform (DFT) matrix. Because of these symmetries, certain computations are repetitive, which means the algorithm can be designed to perform these calculations just once and reuse the results. This significantly reduces the number of complex multiplications required—from $O(N^2)$ in the case of the direct DFT calculation to $O(N)$ for each stage of the FFT.

There are two primary strategies for implementing the FFT: the recursive approach and the iterative approach using for-loops. For my purposes, I am using an iterative implementation known as the Cooley-Tukey FFT algorithm. More specifically, I am applying the Decimation in Time (DIT) version of the Cooley-Tukey FFT, which begins by rearranging the input data points in bit-reversed order before processing them through the subsequent 'butterfly' stages.

The runtime complexity of the FFT is $O(N \log N)$ due to the logarithmic number of stages involved. The $\log N$ factor in the complexity arises because the algorithm recursively divides the computation into two halves at each stage, resulting in a total of $\log_2 N$ stages for an input of size N (when N is a power of two). As a result, when we multiply the linear work done at each stage $O(N)$ by the logarithmic number of stages $\log_2 N$, the overall complexity of the algorithm is $O(N \log N)$.

2.3 I-FFT

The inverse FFT can be calculated using the forward FFT using the following steps:

1. Take the complex conjugate of each data point.
2. Take the FFT of the data.
3. Take the complex conjugate of each data point again.
4. Divide each data point by $\frac{1}{N}$ with N being the total data size.

2.4 Convolution

Convolution can be performed in the time-domain or frequency-domain.

2.4.1 Time Domain Convolution

Time domain convolution uses almost exactly the same code as Lab03, except it removes the time array and sets $\Delta t = 1$.

2.4.2 Frequency Domain Convolution

Frequency domain convolution has a few steps to it.

1. Take the FFT of both data sets.
2. Perform element-wise multiplication on the data sets.
3. Take the IFFT of the resulting data set.

3 Results

3.1 DFT/FFT Timing

The FFT is considerably faster than the DFT. This shouldn't be surprising due to the run-times predicted. It is interesting to note that the FFT is even more efficient than then DFT when it have more data. The plots for the DFT and FFT speeds are shown in Figure 3.

3.2 Time/Frequency Convolution

This result surprised me. I thought frequency convolution would be much faster than the time convolution because it was using the FFT. But surprisingly, they were both pretty close in time all the times I ran them.

4 Discussion and Conclusions

This lab taught me a lot! It was the most intensive lab I have ever done in school up to this point. It taught me how to do research on a topic to be able to understand it well enough to code it up. It also taught me about optimizing code.

With code optimization, I noticed a few things. Not all code optimizations I did necessarily made the code faster. Some code optimizations made the code run more consistently at a certain time. Others made it faster, but less consistent in the timing. Code optimizations aren't always straightforward. Some optimizations can increase the speed with small data sets while slowing the algorithm down as faster data sets. Optimizations need to be done with the thought thought of how much overhead are they adding. Sometimes, due to the overhead, the expected 'optimization' can acaully make the algorithm slower. The best optimizations I found were pre-computing data, in order to not compute redundant values. I didn't get to fully optimize my code, but I got it running in under a second

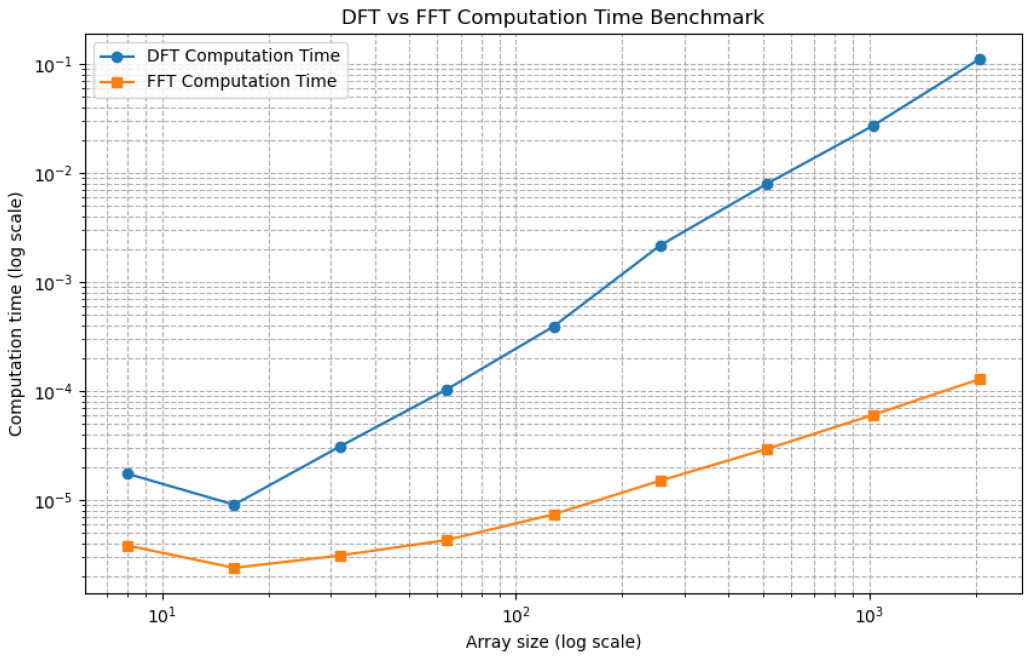


Figure 3: DFT/FFT Time Plot

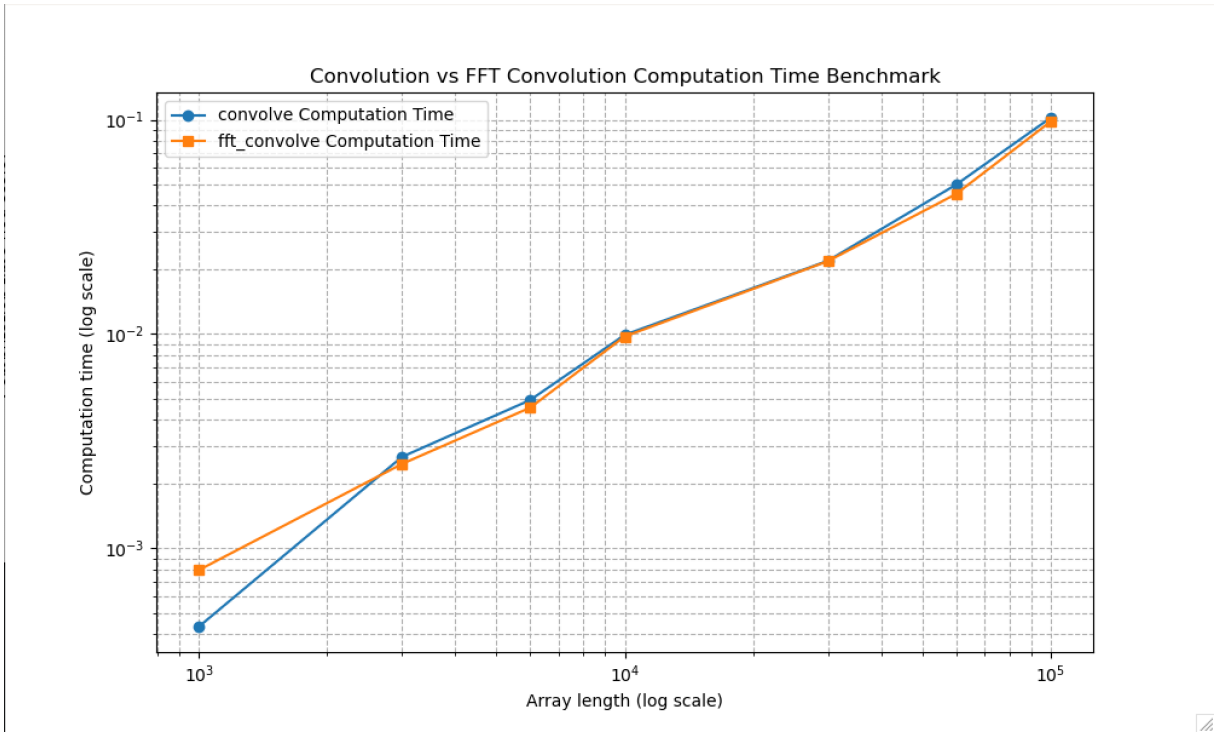


Figure 4: Time/Frequency Convolution Time Plot